Department of Computer Science
Ashoka University

CS-3510: Data Science and Management

Assignment 2 – Part 1

NoSQL Database Design and Querying
with the Yelp Open Dataset

Name: Agriya Yadav
email: agriya.yadav$_ug$2023@*ashoka.edu.in*

Contents

1 Data Acquisition and MongoDB Schema Design **2**
1.1 Collection Structure . . . 2
1.2 Collection Descriptions . . . 2
1.3 Schema Justifications . . . 3
1.3.1 (a) Embed vs. Reference Decisions . . . 3
1.3.2 (b) Read/Write Trade-offs . . . 3
1.3.3 (c) Indexes . . . 4

2 Entity-Relationship (E-R) Diagram **5**

3 Document Schema Diagram **6**

4 Neo4j Property Graph Model **7**

5 MongoDB Queries **9**
5.1 Query 1: Safest and Least-Safe Cities and Business Categories . . . 9
5.2 Query 2: Star Rating Trends Over Time . . . 9
5.3 Query 3: Review Volume vs. Average Star Rating . . . 10
5.4 Query 4: Review Behaviour Across Business Categories . . . 11
5.5 Query 5: Impact of User Tenure on Reviewing Behaviour . . . 11
5.6 Query 6: Elite vs. Non-Elite Users . . . 11
5.7 Query 7: Check-in Activity Patterns vs. Star Ratings . . . 12

6 Neo4j / Cypher Queries **13**
6.1 Query 1: Top 10 Users by Friend Count . . . 13
6.2 Query 2: Top 3 Businesses per State . . . 14
6.3 Query 3: Top 10 Users Across Most Distinct Cities . . . 14
6.4 Query 4: Category-Specific Rating Comparison — Restaurants . . . 15
6.5 Query 5: Reproducing MongoDB Query 1 in Cypher . . . 16

Data Acquisition and MongoDB Schema Design

Collection Structure

The Yelp Open Dataset was subsetted by filtering for five major U.S. cities: **Philadelphia, Tucson, Tampa, Indianapolis, and Nashville**. These cities were selected to represent geographic diversity across the United States and to provide non-trivial data volumes while remaining computationally manageable. The city-based subsetting strategy ensures that all related entities (businesses, reviews, users, tips, checkins) remain interconnected, which is critical for both MongoDB aggregation pipelines and Neo4j graph traversals.

The MongoDB database (`yelp_db`) is organised into **four collections**:

Collection	Records	Source File(s)
`businesses`	47,380	`business.json` + `checkin.json` (embedded)
`users`	819,169	`user.json`
`reviews`	2,640,381	`review.json`
`tips`	347,238	`tip.json`

Table 1: MongoDB collections in `yelp_db`

The source dataset contains six JSON files, but only four were mapped to collections. **Checkins** were embedded directly into business documents (as an array of date strings) rather than kept as a standalone collection. **Photos** (`photo.json`) were excluded entirely because no analytical query in this assignment requires photo data, and including them would add storage overhead without value.

Collection Descriptions

`businesses` — The core entity collection. Each document is identified by `business_id` (unique index) and contains `name`, `city`, `state`, `stars`, `review_count`, `address`, `latitude`, `longitude`, and `is_open`. The `categories` field stores a comma-separated string of business categories. Two nested sub-documents — `attributes` (e.g., WiFi, parking, price range) and `hours` (operating hours by day of week) — are embedded directly. The `checkins` field is an embedded array of ISO-formatted date strings sourced from `checkin.json`. This collection is referenced by `reviews` and `tips` via `business_id`.

`users` — Stores Yelp user account data. Each document is identified by `user_id` (unique index) and includes `name`, `review_count`, `yelping_since`, engagement metrics (`useful`, `funny`, `cool` vote counts), `elite` status, `average_stars`, `fans`, and various compliment counters. The `friends` field is an array of `user_id` strings — a self-referential reference list pointing to other documents in the same collection.

`reviews` — Contains user-written reviews. Each document is identified by `review_id` and includes `user_id` and `business_id` (cross-collection foreign keys), `stars` (1–5 integer), vote counts (`useful`, `funny`, `cool`), full review `text`, and `date`. This is the largest collection and serves as the primary analytical join point between users and businesses.

`tips` — Stores short-form user tips without star ratings. The combination of `user_id`, `business_id`, and `date` serves as a compound key. Includes `text` and `compliment_count`. References both `users` and `businesses` via their respective ID fields.

Schema Justifications

(a) Embed vs. Reference Decisions

Relationship	Decision	Justification
Business ↔ Checkins	Embedded	Checkins have no independent existence — they are never queried separately from their business. Every analytical use (Query 7) accesses them alongside the business document. The data is bounded: each checkin is just a date string. Embedding enables single-document reads without joins.
Business ↔ Attributes/Hours	Embedded (sub-docs)	Descriptive metadata belonging exclusively to one business, always accessed as part of that document, never queried independently. They naturally map to nested objects.
User → Friends	Array of references	Friends are full User entities with large documents. Duplicating entire user documents inside each friend list would create massive redundancy and risk exceeding the 16 MB document limit for well-connected users. Storing an array of `user_id` strings keeps the user document lean while enabling friend-count analytics via `$size`.
Review → Business/User	Cross-collection reference	Reviews are large (text averages ∼550 chars) and grow unboundedly. Embedding them into business documents would make those documents enormous and eventually exceed the 16 MB limit. Reviews are also independently queried (filtered by date, grouped by user). A separate collection with indexed foreign keys is the right model.
Tip → Business/User	Cross-collection reference	Same reasoning as reviews: tips grow unboundedly per business and are queried independently.

(b) Read/Write Trade-offs

Embedding checkins — *Read trade-off*: Positive. Loading a business document automatically loads its entire check-in history in a single fetch. Query 7 (`$size` on the `checkins` array) runs as a single-collection aggregation with no joins required. *Write trade-off*: Slight negative. Every new check-in appends to the array and re-writes the document. However, since the Yelp dataset is a static analytical dataset (no live writes), the read benefit decisively wins.

Referencing reviews — *Read trade-off*: Negative. Queries that need both review content and business details (Queries 2, 4) require a `$lookup` stage, incurring additional I/O. *Write trade-off*: Positive. New reviews are inserted as standalone documents without touching the business document at all — no re-writes, no array growth.

Referencing friends — *Read trade-off*: Negative for friend-detail queries (must resolve each `user_id` to fetch friend profiles), but positive for friend-count queries (`$size` is O(1)). *Write trade-off*: Moderate negative. Adding/removing a friend requires updating the `friends` array on both user documents. However, no MongoDB query in this assignment requires friend-detail resolution — social graph analysis is delegated to Neo4j.

(c) Indexes

Six separate indexes were defined (exceeding the minimum requirement of four) to optimise the specific queries outlined in Section 5.

Field	Collection	Type	Query(s)	Justification
`business_id`	`businesses`	Unique	2, 4, 7	Foreign key join target for `$lookup` from `reviews`. Without this, every join scans 47K documents.
`city`	`businesses`	Standard	1, 2	Enables efficient grouping by city and city+year.
`stars`	`businesses`	Standard	1, 3	Supports range-based bucketing and sorting.
`user_id`	`users`	Unique	6	`$lookup` from `reviews` into `users`. O(log n) over 819K documents.
`business_id`	`reviews`	Standard	2, 4	Every review aggregation that joins to businesses uses this.
`user_id`	`reviews`	Standard	6	Join from reviews to users over 2.6M documents.

Table 3: Indexes defined in `yelp_db`

Entity-Relationship (E-R) Diagram

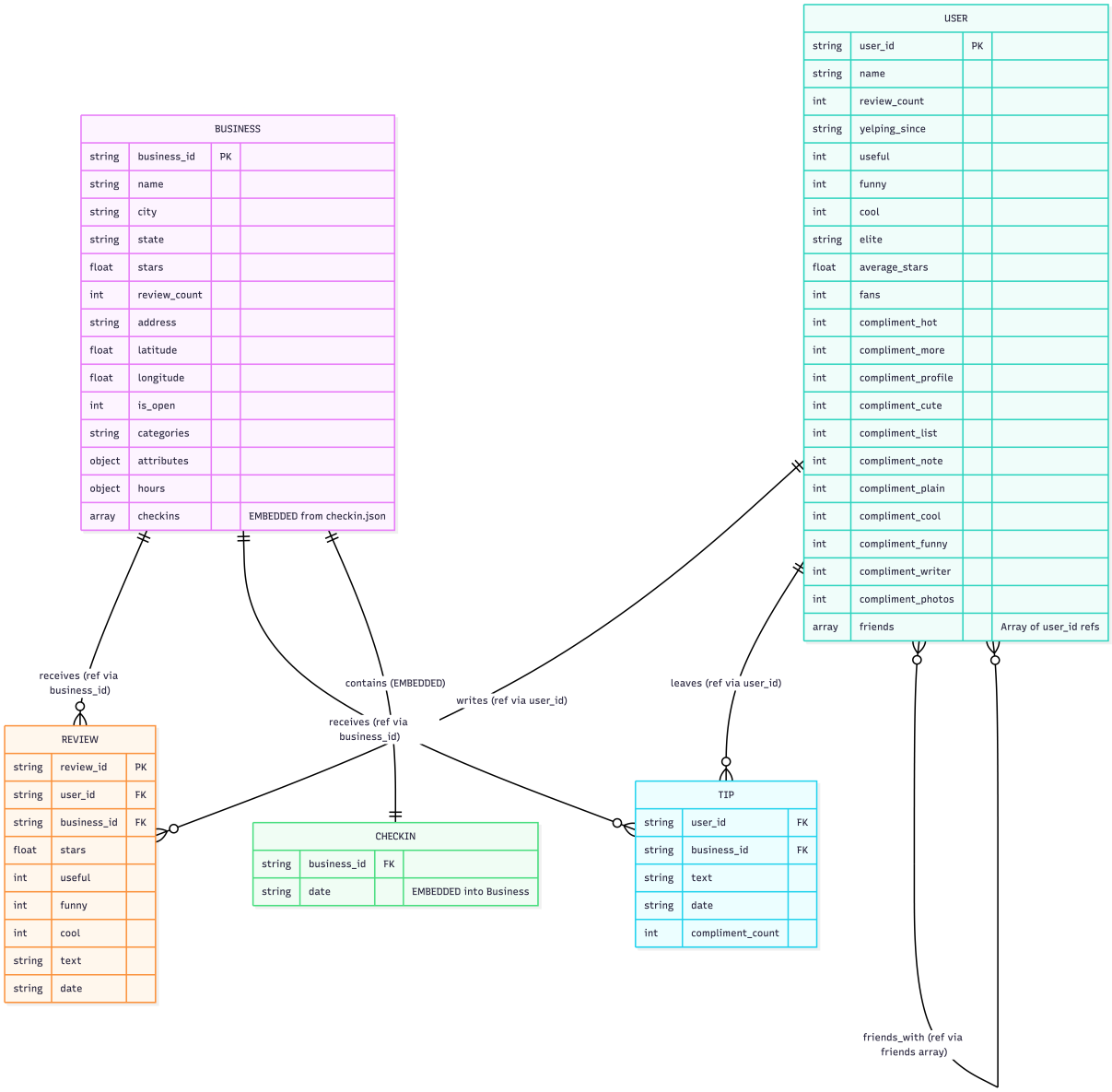

Figure 1: Entity-Relationship diagram for the MongoDB model

The E-R diagram represents the four MongoDB collections as entities. Key features:

- **BUSINESS** ↔ **CHECKIN**: Shown with an *embedded* relationship (double-bordered / labelled connection). Checkins are not a separate collection — they exist as an array within each business document. This is visually distinguished from cross-collection references to emphasise the modelling choice.
- **REVIEW** → **BUSINESS** and **REVIEW** → **USER**: Standard relationship lines with FK labels indicating cross-collection references via `business_id` and `user_id`. Reviews form the primary many-to-many junction between users and businesses.
- **TIP** → **BUSINESS** and **TIP** → **USER**: Same referencing pattern as reviews. Tips connect users to businesses without star ratings.

- **USER ↔ USER (friends_with)**: A self-referential relationship represented by a looping arrow. This models the `friends` array — an array of `user_id` references pointing to other documents in the same `users` collection. This is not an embedding; it is an array of foreign key references.

The distinction between embedded and referenced relationships mirrors the schema design decisions justified in Section 1.3(a). Checkins are embedded because they are bounded and always co-accessed with their business. Reviews and tips are referenced because they grow unboundedly and are independently queried.

Document Schema Diagram

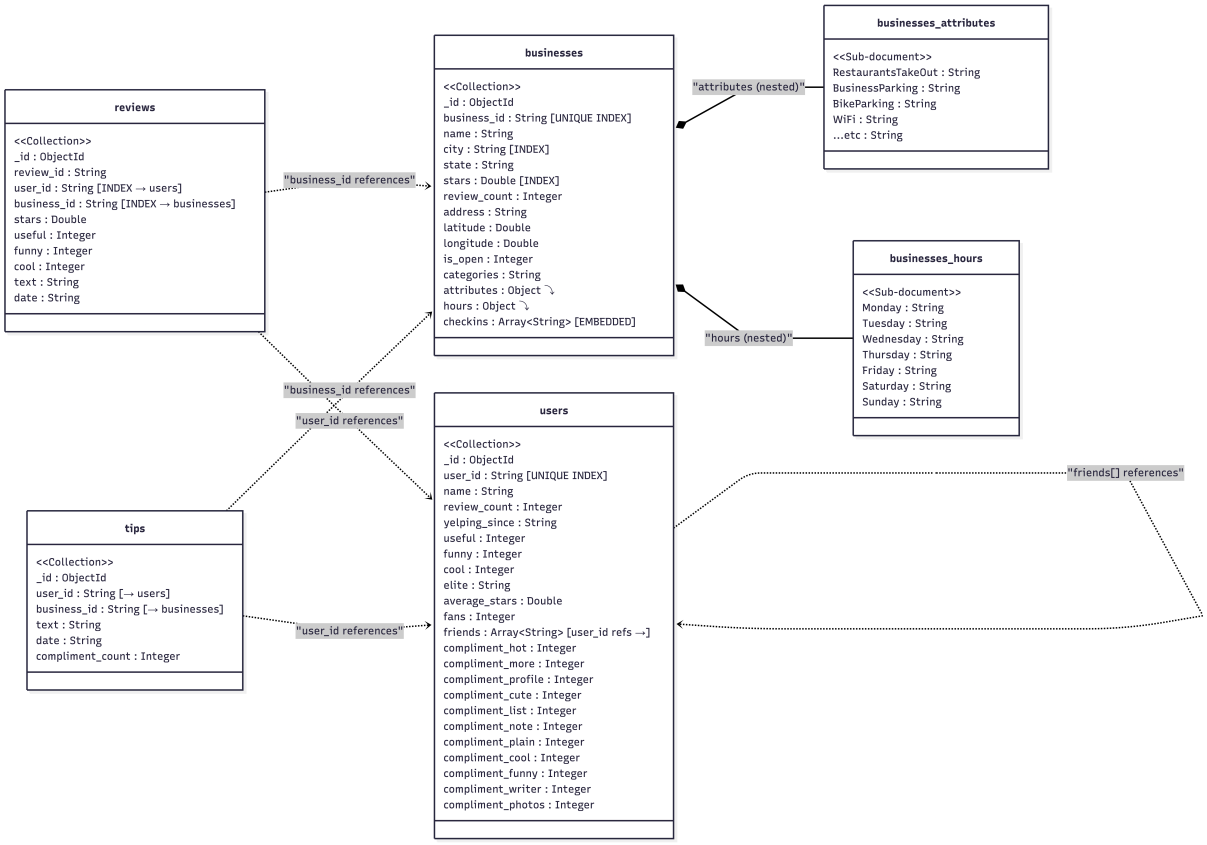

Figure 2: Document schema diagram showing collection structure, fields, types, and references

The document schema diagram shows the internal structure of each collection — fields, data types, nested sub-documents, and cross-collection references.

- `businesses.checkins: Array<String>` — Labelled as EMBEDDED. This array of date strings was sourced from `checkin.json` and merged into business documents during ingestion. The diagram shows it as an internal array, not a separate entity.
- `businesses.attributes` **and** `businesses.hours` — Shown as nested sub-documents (child classes connected with composition arrows). `attributes` contains variable key-value pairs (e.g., `RestaurantsTakeOut`, `WiFi`, `BikeParking`), while `hours` con-

tains day-of-week strings. These are objects nested within the business document, not separate collections.

- `users.friends: Array<user_id refs>` — Shown as an array field with a reference arrow pointing back to the `users` collection. Friend data is not embedded user documents — it is an array of string references.
- `reviews.business_id [INDEX]` **and** `reviews.user_id [INDEX]` — Shown with reference arrows pointing to `businesses` and `users` respectively, labelled with INDEX to indicate both fields are indexed for efficient `$lookup` joins.
- **Index annotations** are marked directly on fields: `businesses.business_id [UNIQUE INDEX]`, `businesses.city [INDEX]`, `businesses.stars [INDEX]`, `users.user_id [UNIQUE INDEX]`, `reviews.business_id [INDEX]`, `reviews.user_id [INDEX]`.

Neo4j Property Graph Model

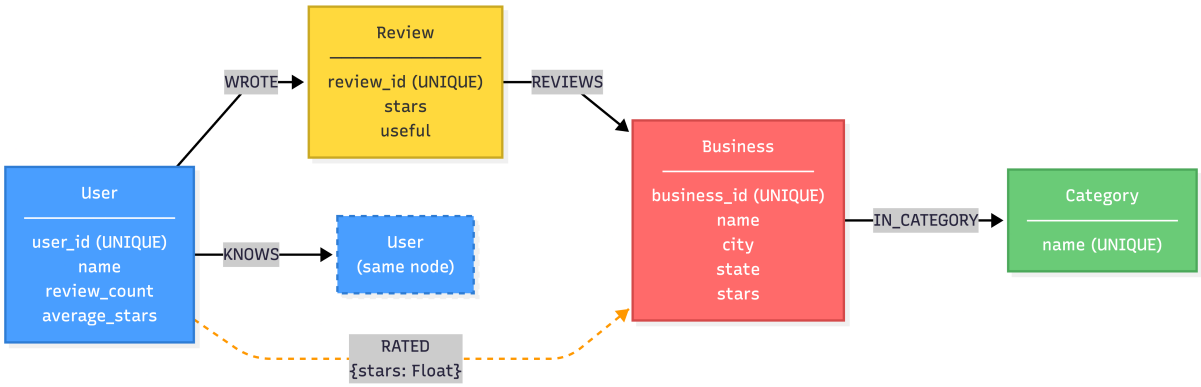

Figure 3: Neo4j property graph model diagram

Node Labels and Properties

Node Label	Properties	Constraint
`:User`	`user_id`, `name`, `review_count`, `average_stars`	`user_id` UNIQUE
`:Business`	`business_id`, `name`, `city`, `state`, `stars`	`business_id` UNIQUE
`:Review`	`review_id`, `stars`, `useful`	`review_id` UNIQUE
`:Category`	`name`	`name` UNIQUE

Relationship Types

Relationship	From	To	Properties	Description
`WROTE`	User	Review	—	User authored a review
`REVIEWS`	Review	Business	—	Review is about a business
`RATED`	User	Business	`stars: Float`	Shortcut denormalization edge
`IN_CATEGORY`	Business	Category	—	Business belongs to a category
`KNOWS`	User	User	—	Social friendship (self-referential)

Modelling Choices

Review as a node, not an edge: Reviews carry rich properties (`stars`, `useful`) and sit between two entity types. Modelling Review as a node allows properties to be attached naturally and enables flexible multi-hop traversal patterns. If reviews were edges, attaching multiple properties would be less clean.

RATED shortcut relationship: The `RATED` edge is a denormalized shortcut from User directly to Business, storing the `stars` property. This avoids the two-hop traversal `(User)-[:WROTE]->(Review)-[:REVIEWS]->(Business)` for common queries. The trade-off is slight storage redundancy.

Category as a separate node: In MongoDB, categories are stored as a comma-separated string requiring `$split` and `$unwind` for analysis. In Neo4j, each category is a separate node with `IN_CATEGORY` edges, naturally handling the many-to-many relationship as simple edge traversal.

KNOWS relationship (directed): The source data provides one direction (user A lists user B as a friend), so edges are directed. Cypher queries account for this by matching outgoing edges `(u)-[:KNOWS]->(f)`.

Data That Does Not Map Naturally to a Graph

- **Checkins** — A time-series list of timestamps. Graphs do not have a natural ordered sequence structure; modelling each checkin as a node would create millions of nodes with minimal traversal value. Handled as the embedded array in MongoDB.
- **Business attributes/hours** — A sparse, schema-less key-value set. Graph nodes expect consistent property sets. These were left out of the graph model.
- **Review text** — Free-text has no graph traversal value. Excluded from Review nodes to keep the graph lightweight. Text analysis is performed in MongoDB.

MongoDB Queries

Query 1: Safest and Least-Safe Cities and Business Categories

Part A — By City (businesses with ≥20 reviews)

City	Avg Stars	Total Reviews	Business Count
Philadelphia	3.69	863,550	7,148
Indianapolis	3.66	307,878	3,257
Nashville	3.65	405,627	3,355
Tampa	3.63	390,233	3,909
Tucson	3.59	335,105	3,853

Philadelphia leads at 3.69 average stars; Tucson ranks lowest at 3.59. The narrow range (0.10 stars) indicates all five cities are generally well-rated. The `$match: { review_count: { $gte: 20 } }` filter ensures statistical significance by excluding businesses with too few reviews.

Part B — By Category (top 5 safest and least-safe)

Rank	Safest Categories	Avg Stars
1	Barre Classes	4.71
2	Home Window Tinting	4.63
3	Challenge Courses	4.61
4	Event Photography	4.61
5	Dog Walkers	4.60

Rank	Least-Safe Categories	Avg Stars
1	Television Service Providers	1.63
2	Utilities	1.77
3	Internet Service Providers	1.94
4	Truck Rental	2.18
5	Banks & Credit Unions	2.20

Table 4: Top and bottom categories by average star rating

The highest-rated categories are niche, passion-driven services (fitness, photography, pet care) where customers self-select and providers are intrinsically motivated. The lowest-rated categories are utility/necessity services (telecom, banking, truck rental) — industries where customers have little choice and friction is inherent to the service model.

Query 2: Star Rating Trends Over Time

Part A — City Trends

The aggregation joins reviews with businesses to extract year and city, then computes average ratings per city-year combination.

- **Early years (2005–2007)**: Inflated ratings (3.9–4.7) due to very few reviews — enthusiastic early adopters creating positive selection bias.
- **Stabilisation (2010–2019)**: Ratings converge around 3.7–3.9 as review volume grows into tens of thousands per city per year.
- **COVID year (2020)**: Review counts drop sharply (Indianapolis: 50,236 in 2019 → 30,620 in 2020) but average ratings increase slightly (3.83 → 3.91). People only visited places they trusted during lockdowns.
- **2022 truncation**: Counts dramatically lower (Indianapolis: 1,829 vs 34,140 in 2021) — the dataset ends mid-year.

Part B — Category Trends (early 3 years vs. late 3 years)

Category	**Early Avg**	**Late Avg**	**Change**
Strongest upward trends			
Tiki Bars	3.30	4.12	+0.81
Limos	3.04	3.74	+0.70
Home Cleaning	3.19	3.86	+0.67
Tea Rooms	3.77	4.41	+0.64
Beaches	3.54	4.18	+0.63
Strongest downward trends			
Hospitals	3.46	2.40	−1.06
Walk-in Clinics	3.76	2.71	−1.04
Banks & Credit Unions	3.10	2.07	−1.03
Car Rental	3.04	2.19	−0.85
Packing Supplies	3.83	2.98	−0.84

Table 5: Category rating trends (early vs. late years)

Experiential categories (Tiki Bars, Limos, Tea Rooms) trend upward as standards improved over time. Healthcare and essential services (Hospitals, Banks, Car Rental) trend sharply downward — as Yelp's user base grew, platforms increasingly became outlets for venting systemic service frustrations.

Query 3: Review Volume vs. Average Star Rating

Review Count Bucket	**Avg Stars**	**Business Count**
0–9	3.59	14,608
10–49	3.55	21,986
50–99	3.66	4,980
100–499	3.79	5,181
500–999	3.97	489
1000–4999	4.07	134
5000+	4.50	2

There is a clear **positive correlation**: businesses with more reviews tend to have higher average ratings. This is explained by **survivorship bias** — poorly-rated businesses close, so only high-quality businesses accumulate large review counts. *Practical implication*: raw star ratings should always be contextualised with review count.

Query 4: Review Behaviour Across Business Categories

Category	Avg Stars	Avg Useful	Avg Text (chars)	Total Reviews
Counseling & Mental Health	3.06	4.52	906	1,370
Lawyers	3.39	4.51	575	1,809
Insurance	2.71	3.87	642	2,497
Colleges & Universities	3.21	3.79	838	1,913
Property Management	2.53	3.46	877	6,243
Apartments	2.67	2.86	961	15,548

Counseling & Mental Health and **Lawyers** top the useful-votes ranking — high-stakes categories where readers seek detailed guidance before making decisions. These same categories have the longest reviews (906 and 838 avg chars). **Low-rated categories** (Insurance 2.71, Apartments 2.67, Property Management 2.53) reflect structural dissatisfaction — services people use out of necessity, not choice. The negative sentiment is systemic, not business-specific.

Query 5: Impact of User Tenure on Reviewing Behaviour

Tenure (years)	Avg Stars	Avg Useful Votes	Total Users
0–1	2.79	0.16	908
2–4	3.30	4.24	72,811
5–9	3.65	22.49	401,631
10–14	3.74	91.39	321,286
15+	3.76	431.73	22,533

Useful votes increase **2,700**× from the 0–1 year cohort to the 15+ year cohort (0.16 → 431.73). Veteran users contribute disproportionately to platform value despite being a small minority. Stars increase slightly with tenure (2.79 → 3.76) — newer users give extreme ratings; experienced users calibrate more moderately over time.

Query 6: Elite vs. Non-Elite Users

Metric	Non-Elite	Elite	Difference
Mean star rating given	3.70	3.98	+0.28
Mean review character length	507	780	+54%
Mean useful votes per review	0.82	2.11	+2.6×
Total reviews	1,874,725	765,646	—

Elite users write **54% longer reviews** and receive **2.6× more useful votes per review**. Only ∼6.5% of users hold elite status (52,973 / 819,169), yet they account for **29% of all reviews** (765,646 / 2,640,371). Elite status is selective and correlates strongly with review quality and platform engagement.

Query 7: Check-in Activity Patterns vs. Star Ratings

Part A — Overall check-in volume vs. average stars

Checkin Bucket	Avg Stars	Business Count
0–9	3.59	18,865
10–49	3.58	13,820
50–199	3.61	8,735
200–999	3.69	5,052
1000+	3.89	908

Positive correlation: businesses with more check-ins tend to have higher ratings (3.59 → 3.89). Popular businesses attract more foot traffic and maintain higher quality.

Part B — By category (top categories by avg check-ins)

Category	Avg Checkins	Avg Stars	Business Count
Airports	2,838	3.10	57
Shopping Centers	1,371	3.54	80
Gastropubs	680	3.85	189
Cheese Shops	654	4.26	55
Airlines	589	2.45	59
Wineries	538	4.13	73
Pubs	468	3.69	583
Breweries	460	4.09	262

Does the relationship vary by category? Yes. Airports and Airlines break the overall positive correlation entirely (high check-ins, low stars — captive customers, not satisfied ones). Gastropubs, Wineries, and Breweries confirm it (high check-ins *and* high ratings), suggesting genuine repeat patronage. The relationship is highly category-dependent.

This also validates the **embedding decision**: check-in data stored on the business document made this a single-collection scan with no joins.

Neo4j / Cypher Queries

Query 1: Top 10 Users by Friend Count

Name	Friend Count	Review Count	Mean Star Rating
Walker	14,995	585	3.91
Ruggy	12,395	2,434	3.98
Randy	11,026	3,315	3.77
Scott	10,366	789	3.86
Steven	10,072	1,371	3.62
Katie	9,390	1,825	4.23
Rodney	8,809	1,237	4.09
Stephanie	8,087	2,000	3.85
Emi	8,028	1,926	4.33
Frank	7,945	1,006	4.08

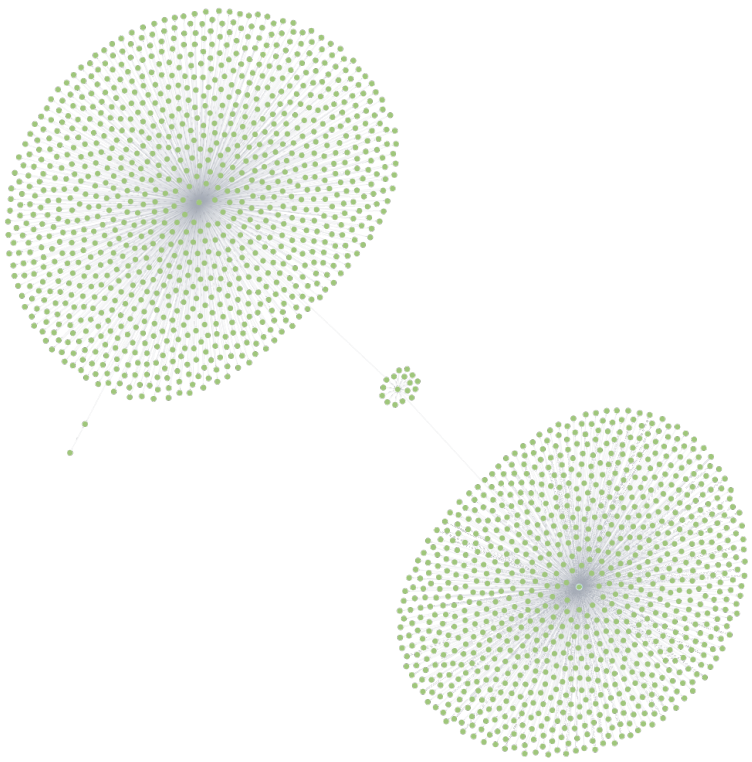

Figure 4: Social network graph of the top 10 most-connected users. The two large clusters represent Walker's and Ruggy's friend networks.

There is no strong correlation between friend count and review count. Walker leads with 14,995 friends but only 585 reviews — highly social but not a prolific reviewer. Conversely, Randy has the most reviews (3,315) with fewer friends. Social connectivity and review activity are independent dimensions. Mean star ratings cluster tightly (3.62–4.33), suggesting social influence does not systematically bias rating behaviour.

Query 2: Top 3 Businesses per State

St.	Business Name	City	Top Categories	Stars	Rev.
AZ	Let's Sweat	Tucson	Yoga, Fitness, Cycling	5.00	59
AZ	Premier Pest Solutions	Tucson	Local Services, Pest Control	5.00	56
AZ	Tremendez Jewelry & Repair	Tucson	Jewelry, Watch Repair	4.98	55
FL	AllVitae Health & Chiro.	Tampa	Massage, Skin Care, Health Coach	5.00	67
FL	Gerardo Luna Photographs	Tampa	Photography, Event Planning	4.98	51
FL	Tampa Bay Rum Company	Tampa	Food, Distilleries	4.96	57
IN	Alvin Lui	Indianapolis	Arts, Entertainment, Magicians	5.00	50
IN	Brick & Mortar Barber Shop	Indianapolis	Barbers, Beauty & Spas	4.97	72
IN	kOMpose	Indianapolis	Life Coach, Yoga, Fitness	4.97	58
PA	Guldner's Collision Service	Philadelphia	Automotive, Body Shops	5.00	57
PA	Campbell Jewelers	Philadelphia	Jewelry, Jewelry Repair	5.00	55
PA	Sugar Bar Salon	Philadelphia	Hair Removal, Beauty & Spas	5.00	55
TN	Taylor Home Solutions	Nashville	Carpeting, Home Services, HVAC	5.00	50
TN	Music City Cats	Nashville	Pet Services, Dog Walkers	5.00	54
TN	Walls Jewelry Repairing	Nashville	Jewelry, Watch Repair	5.00	114

Table 6: Top 3 businesses per state by average star rating ($\geq$50 reviews)

All top businesses have near-perfect ratings (4.96–5.00) with modest review counts (50–114), reflecting **statistical fragility** — a small number of very satisfied customers can maintain a perfect score. Category diversity across states is notable: AZ favours wellness/-fitness, FL hospitality and food, IN entertainment/grooming, PA automotive/jewellery, TN home services.

Query 3: Top 10 Users Across Most Distinct Cities

All 10 users covered all 5 cities in the subset. Since the dataset was subsetted to exactly 5 cities, 5 is the ceiling — a dataset limitation, not a meaningful differentiation. On the full Yelp dataset, this query would effectively identify travelling reviewers and map food tourism patterns across states.

The differentiator is **per-city mean rating variation**:

- **David**: rates Tampa highly (4.50) but Indianapolis lower (3.60) — possible preference for Tampa's business landscape.
- **Olivier**: rates Tucson at 4.00 but Indianapolis at 2.50 — significant city-level bias.
- **S**: consistently generous rater (3.75–5.00 across all cities). Note: truncated name is a data quality issue in the Yelp source.

Query 4: Category-Specific Rating Comparison — Restaurants

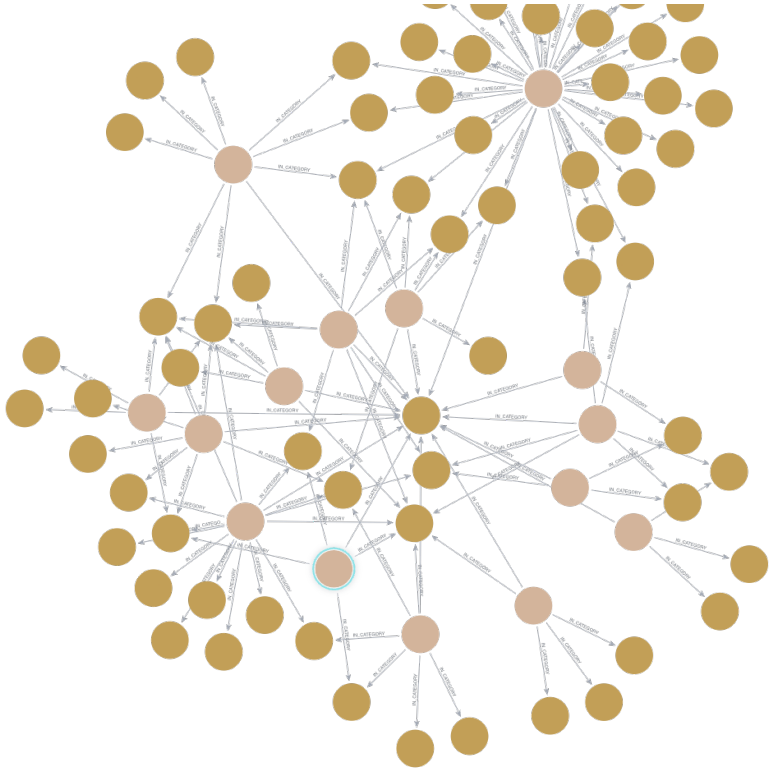

Figure 5: User → Review → Business → Category traversal graph for the Restaurants category

User	Restaurants Reviewed	User Avg (Restaurants)	User Overall Avg	Deviation
Michelle	714	4.01	4.08	−0.07
Boon	631	3.90	3.89	+0.01
Bill	608	4.05	4.07	−0.02
Brett	556	4.19	4.24	−0.05
michelle	547	3.85	3.88	−0.03
Karen	538	3.53	3.62	−0.10

Table 7: Users who reviewed ≥5 Restaurants, compared against their own overall average

Most users rate restaurants **slightly lower** than their overall average (largely negative deviations), indicating a more critical lens applied to food specifically. **Karen** (−0.10) is the most category-critical. **Boon** is the most consistent (+0.01). **Michelle** reviewed 714 restaurants — an extreme power user — yet her deviation is a modest −0.07, suggesting her restaurant behaviour closely mirrors her general reviewing style.

Query 5: Reproducing MongoDB Query 1 in Cypher

City	Avg Stars	Total Reviews	Business Count
Philadelphia	3.69	895,422	7,288
Indianapolis	3.64	320,311	3,330
Nashville	3.64	416,408	3,414
Tampa	3.63	405,732	3,988
Tucson	3.58	352,978	3,963

Results match closely: rankings are identical (Philadelphia first, Tucson last). Minor differences in total review counts (MongoDB: 863,550 vs. Cypher: 895,422 for Philadelphia) arise because MongoDB used the pre-stored `review_count` field on the business document, while Cypher counted actual Review nodes present in the ingested graph subset.

Which database is better suited? MongoDB is better for this query. It is a pure aggregation on a single collection — no relationship traversal is needed. The `$match` → `$group` → `$sort` pipeline is highly optimised for flat, tabular aggregations. Cypher must traverse `(Business)<-[:REVIEWS]-(Review)` edges to count reviews, which is elegant but slower for aggregate-only analytics. The graph model's strength shows when queries need relationship traversal — friend networks, multi-hop paths, category recommendations — not flat aggregations.

Appendix: Query Scripts

All MongoDB queries are provided in `MongoDB_Queries.txt`.
All Neo4j/Cypher queries are provided in `Neo4j_Cypher_Queries.txt`.

AI Usage

Claude (Anthropic) was used to assist with schema design reasoning, query writing, and report structuring. Full conversation log:
`https://claude.ai/share/dbff8110-f2ef-45b1-a294-e9bed37d1521`